

Domain-Specific RAG Chatbot for PDF Question Answering

Overview

The application allows users to upload PDF documents, process them into local vector embeddings, and interact via an intuitive Streamlit chat interface powered by LangChain and the Groq API.

1. Project Structure & Environment Setup

Document loading, vector storage, prompting, pipeline orchestration, and the user interface.

- **Dependencies:** Managed via requirements.txt, including core libraries such as Streamlit, LangChain components, HuggingFace embeddings, FAISS for vector search, and PyPDF for document parsing.
- **Environment Configuration:** A .env file securely stores the required GROQ_API_KEY.
- **Virtual Environment:** Set up locally using Python's venv to isolate dependencies.

2. Core Implementation Components

Document Processing (document_loader.py)

- Handles temporary file storage for uploaded PDFs.
- Extracts raw text while preserving original filenames and page metadata (with 1-based indexing for clarity).
- Splits large documents into manageable text chunks using RecursiveCharacterTextSplitter

Vector Storage (vector_store.py)

- Utilizes the sentence-transformers/all-MiniLM-L6-v2 embedding model via HuggingFace.
- Builds and maintains a local FAISS vector index to enable fast similarity searches over document chunks.

Guardrails & Prompting (prompt.py)

- Enforces strict grounding rules to prevent hallucinations or reliance on external knowledge.
- Configures a fallback response whenever requested information is absent from the provided context:

"I could not find this information in the uploaded documents."

RAG Pipeline (rag_pipeline.py)

- Combines the FAISS retriever, context formatter, prompt template, and Groq's llama-3.3-70b-versatile LLM.
- Formats retrieved context documents to prominently display source filenames and page numbers.

User Interface (app.py)

- Built with Streamlit to provide a responsive, multi-page layout.
- Features a sidebar for document management and index clearing, alongside a chat history interface that logs user queries, generated answers, and expandable source attributions.

3. Execution & Testing

1. **Launch the Server:** Run the application locally in the terminal via `streamlit run app.py`.
2. **Upload & Index:** Upload target PDF files through the sidebar and click **Process Documents**.
3. **Validation:** Test the chatbot with both in-domain queries and out-of-domain questions to verify that the fallback guardrails trigger correctly.